# Notes (Week 7 Monday)

These are the notes I wrote during the lecture.

```
A relation ℛ such that:
  (s,s') ∈ ℛ

this means that, if we start from
the initial state s,
a *possible* final result of
running the program is s'


We're using "state machines"
  and "transition systems"
  as synonyms

The relations we've been looking
 at so far (e.g. in assignment 1)
 are "big-step" relations:

  (s,s') ∈ ℛ
one jump through the relation
takes us immediately from
initial to final state

In transition systems,
 we consider "small-step" relations
 one step through the relation
 represents one computation step
 which isn't necessarily the whole
 computation


Bounded counter system:

  S = {0,1,...,99,overflow}

  Transition relation

  → ⊆ S × S

  → = {(x,x+1) : x ≤ 98}
      ∪ {(99,overflow)}
      ∪ {(overflow,overflow)}

Very often, when we model programs,
```

```
  it's enough to track the values
  of variables (let's say)

    s → s'

Also very often, you distinguish
 between (internal) state
 and externally observable events.
You'd put the events on the labels,
 and the internal stuff in the states.

 void hi() {
   printf("Hello world!");
 }

 void hello() {
   int x = 1;
   if(Math.Random() < .5)
     printf("Hello world!");
   else {
     x++;
     printf("Hello world!");
   }
 }

We might model these as transition systems
  where the labels are:

    print_hello    for the printf event
    τ or no label  for unobservable events

The definition of determinism in an
  unlabelled transition system is:

 for every state s, there is at most one s'
 such that s → s'

For *labelled* transition systems:

 for every state s and label α, there is at
 most one s' such that

        s --α--> s'

 (0,0) → (0,3) → (3,0) →
 (3,3) → (5,1) → (0,1) → (1,0)
 (1,3) → (4,0)


The counter transition system
```

```
Here are some runs (from 0):

  0,1,2,3,4,5
  0
  0,1,2,3,...,99,overflow,overflow,..
  ^ the last one is the only maximal run

If R is a relation,
then R* is the
  *reflexive and transitive closure of R*

Defined inductively by the following rules:

  (x,x) ∈ R*

  If (x,y) ∈ R  and (y,z) ∈ R*
  then (x,z) ∈ R*

If R is a transition relation, then
 x R* y means
 "in 0 or more steps, we can get from x to y"

Safety and liveness properties are
 not moral judgements

 Safety "something bad will never happen"
 Liveness "something good will happen"

Mostly ignore the words "bad" and "good"

 I will come home tonight,
  and find a chocolate cake in my fridge

 I will come home tonight,
  and find something super gross in my fridge

^ both of these are liveness properties

In order to have a useful system,
 you want it to satisfy both safety
 and liveness properties.

 A system that does nothing
  satisfies every safety property.

 If you only specify a liveness property,
  you can't rule out undesired behaviour

Q: How is this a safety property?

  If the program terminates, then y=x!
```

```
A: it can be equivalently stated as follows:

  The program will never reach a final state
  where y ≠ x!


Safety:
  A safety property can be falsified
   by a finite prefix of a behaviour

Liveness:
 It is never too late to satisfy the
 liveness property.
 Any finite prefix of a behaviour
  can be extended into one that satisfies
  the liveness property.

When I come home,
  I'll drop on the couch and drink a beer

1. this is a liveness property
When I come home,
  I'll *eventually* drop on the couch and drink a beer

2. this is a safety property
When I come home,
  I'll *immediately* drop on the couch and drink a beer
```